

1. Project Initialization & Dependency Management

After cloning the repository, the first step was to prepare the environment for both the Frontend and Backend.

- **Installing Packages:** We ran `npm install` in the `server` directory to install essential libraries like `express` (web server), `pg` (PostgreSQL client), and `dotenv` (environment variable management).
- **Environment Setup:** We ensured a `.env` file was present to store sensitive configurations like database credentials and port numbers.

2. Database Deployment via Docker

Instead of installing PostgreSQL directly on Windows, we used a Docker container to host the database.

- **Containerization:** We launched the PostgreSQL 18 container, which keeps the database isolated from the rest of your system.
- **Port Mapping:** We verified via `docker inspect` that the container's internal port (5432) was correctly mapped to your computer's port (5432). This allows the Node.js backend to "talk" to the database inside Docker.

3. Environment Configuration (.env)

We had to tell the Backend exactly where the database was and how to secure user sessions.

- **Database URL:** We configured the `DATABASE_URL` using `localhost`. Since your Backend runs on your Windows host and the DB is mapped to a Windows port, `localhost` acts as the bridge.
- **Security (SECRET_KEY):** We identified a crash caused by a missing `SECRET_KEY`. We added a secret string to the `.env` file, which the Backend uses to sign and verify JWT (Login) tokens.

4. Database Schema Migrations

A running database is empty by default. We had to create the structure (tables) required for LPMS.

- **Running Migrations:** By executing `npm run migrate`, we triggered a script that automatically created the `auth_principals` table and other necessary schemas.
- **Verification:** This step ensured that when you try to log in, the Backend actually has a table to check your username and password against.

5. Frontend-Backend Integration (Vite Proxy Fix)

This was the most critical step to fix the "500 Error" and "Proxy Error" you saw in the browser.

- **Port Correction:** Your `vite.config.ts` was originally pointing to port 5000. We corrected this to **Port 3000** to match your actual Backend server.

- **IPv4 Stability:** We changed the target from `localhost` to `127.0.0.1`. This prevents a common Windows issue where the frontend tries to use IPv6 while the backend is listening on IPv4, which causes a "Connection Refused" error.

6. Implementation of Debugging Tools

To see why things were failing, we added better error reporting.

- **Unhandled Rejection Hook:** We added a listener in `index.js` to catch "silent" database errors and print them clearly in the terminal.
- **Nodemon Monitoring:** We used `nodemon` to watch for file changes, ensuring that every time we updated the `.env` or config files, the server restarted automatically with the new settings.

Current Status: Your system is now fully "wired" together. The Frontend talks to the Backend, the Backend talks to the Database, and the Security keys are in place.